

架构决策记录ADR

ADR-001：采用模块化垂直架构 (Modular Vertical Architecture)

状态

已接受

背景

我们团队此前从未有过Web开发的经验，且该项目开发周期紧迫。若采用传统的水平分层架构，多人频繁修改同一代码模块可能导致严峻的 Git 合并冲突。此外，由于成员对业务边界认知可能存在偏差，极易出现“协作阻塞”现象，增加项目交付风险。

决策

我们决定采用模块化垂直架构。将系统按业务功能进行垂直切分，目前暂定是用户管理、任务管理、标注界面、裁定界面这四部分。每位成员负责一个完整模块的端到端开发，包括前端页面、后端逻辑及数据库表设计，模块间通过预定义的 API 接口进行轻量化交互。

理由

- 低耦合协作**：垂直切分确保了代码物理隔离，极大减少了多人协作时的 Git 冲突成本。
- 并行效率最大化**：成员拥有独立的业务空间，无需等待他人完成底层通用类即可推进业务。
- 责任明确**：每个模块有唯一的负责人，便于 Bug 追踪和课程作业的个人绩效评定。

后果

- 正面影响**：开发并行度高，代码合并冲突减少，交付压力得到缓解。
- 负面影响**：可能存在少量公共工具类的重复实现；模块功能划分需要前期仔细商讨；对前期 API 契约设计的准确性要求较高。
- 需要关注的风险**：必须在初期强制统一 API 返回格式和全局异常处理，否则后期联调会面临巨大重构压力。

AI 辅助记录

- AI 初稿内容摘要**：建议使用经典 MVC 三层架构，侧重于代码复用和逻辑分层，未考虑团队协作冲突。
- 人工修订内容**：改为“模块化垂直架构”，强调按业务功能切分以匹配四人团队的协作模式，以此提高开发效率。
- 修订理由**：AI 忽略了开发效率这一关键质量属性在短工期项目中的优先级。垂直架构虽牺牲了部分复用性，但换取了极高的协作并行度，可以极大减少因认知偏差造成的项目进度拖延。

ADR-002：采用 Vue3 + Spring Boot 前后端分离架构

状态

已接受

背景

项目需要开发一个交互复杂的标注系统，涉及动态表单、实时预览等功能。我们团队成员具备Java和基础前端能力，技术选型需考虑当前的主流趋势和开发效率。项目需在本地服务器或云主机部署，且需要快速迭代。

决策

我们决定采用 Vue3 作为前端框架，配合 Spring Boot 作为后端框架，通过 RESTful API 进行数据交互。构建工具使用 Vite，UI组件库选用 Element Plus。

理由

- **技术成熟度**：Spring Boot 是 Java 领域最成熟的后端框架，生态极其完善；Vue3 已成为前端通用工具，技术成熟稳定。
- **开发体验**：Vite构建工具启动速度极快，热更新无延迟，极大提升学生开发体验。
- **生态丰富**：两者都有海量的开源组件和文档，遇到Bug能快速找到解决方案，降低课程作业的失败风险。
- **前后端分离**：符合现代Web开发标准，便于前后端并行开发，且界面美观度高，利于课程答辩展示。

后果

- **正面影响**：开发效率高，界面交互流畅，系统扩展性好。
- **负面影响**：增加了跨域配置和静态资源部署的复杂度；首屏加载需优化。
- **需要关注的风险**：需严格管理 Node.js 和 Java 的版本依赖，避免因环境差异导致系统不兼容的问题。

AI 辅助记录

- **AI 初稿内容摘要**：推荐了 Vue3 和 Spring Boot 技术栈，但未结合我们团队的开发能力进行考量，甚至给出了较为前沿的技术栈，学习压力过大。
- **人工修订内容**：结合我们团队的特点，考虑到传统技术栈具有丰富的生态，简单易上手，最终锁定 Vue3 和 Spring Boot 的前后端框架。
- **修订理由**：AI的推荐较为通用，缺乏针对课程设计场景的考量。我们需要的是成功率高的方案，而非追求最前沿但不稳定的实验性技术。本次课程设计强调高确定性交付，盲目追求实验性技术会导致团队在环境搭建和语法磨合上浪费过多精力。

ADR-003：采用 MySQL 单库存储方案

状态

已接受

背景

项目为课程设计，我们小组预计，本平台用户量<1000，标注数据量<10万条，无高并发场景。部署环境为校内服务器或低配云主机。团队缺乏运维分布式数据库的经验，需在短时间内完成数据库设计与开发。

决策

我们决定使用单一的 MySQL 关系型数据库实例，配合 InnoDB 存储引擎。不引入Redis缓存集群或NoSQL数据库，除非有明确的性能瓶颈。

理由

- **数据一致性**：标注系统涉及复杂的业务关系，MySQL 的 ACID 事务能保证数据强一致性。
- **开发成本低**：无需搭建复杂的主从复制或分库分表中间件，单机部署简单，符合学生团队运维能力。
- **工具链完善**：MySQL 相应工具成熟，便于团队成员查看和调试数据。
- **完全满足需求**：对于课程级别的数据量，MySQL 的性能绰绰有余，引入分布式数据库属于过度设计。

后果

- **正面影响**：架构简单稳定，数据安全有保障，开发人员能专注于业务逻辑而非数据库运维。
- **负面影响**：如果未来希望对平台性能进行提升，或者用户与标注数据量持续增加，单库将成为瓶颈。
- **需要关注的风险**：为避免机器宕机导致服务不可用，需定期备份数据以防误删。

AI 辅助记录

- **AI 初稿内容摘要**：直接建议使用MySQL，未提及数据规模、并发量以及与NoSQL方案的对比。
- **人工修订内容**：补充了无高并发、学生运维能力有限的客观条件，并明确排除了 Redis或者MongoDB 的复杂方案。
- **修订理由**：AI倾向于推荐通用的技术，但忽略了课程项目并不是生产环境，在数据量很小的情况下，保持架构简单是最重要的工程原则。

ADR-004：采用 JWT 无状态身份认证方案

状态

已接受

背景

系统采用 Vue3 与 Spring Boot 前后端分离架构，存在跨域请求。传统的 Session+Cookie 机制在跨域场景下配置繁琐，且依赖后端服务器存储 Session 状态，不利于后续可能的容器化扩展及本地环境模拟。

决策

我们决定使用 JWT 实现身份认证。后端在用户登录后签发自包含的 Token，前端将其存储并在请求头中携带，后端通过拦截器进行无状态校验。

理由

- **跨域友好**：天然支持 RESTful API 和跨域访问，无需处理 Session 共享问题。
- **降低内存压力**：服务器不存储会话信息，适合低配的课程部署环境。
- **开发生态成熟**：Spring Security + JWT 方案已有极高质量的开源模板，便于学生快速集成。

后果

- **正面影响**：认证逻辑解耦，前后端对接极其简单。
- **负面影响**：Token 一旦签发，服务端在有效期内始终生效，无法强制注销。
- **需要关注的风险**：Token 的存储易受黑客攻击，需对输入进行严格过滤。

AI 辅助记录

- **AI 初稿内容摘要**：给出了 JWT 的代码实现步骤，将其描述为完美方案。
- **人工修订内容**：补充了关于安全性的权衡，明确指出了 Token 无法即时失效的负面影响及黑客防范方案。
- **修订理由**：AI 往往只关注技术的可实现性，而忽略了安全性与可控性的权衡。对于涉及用户权限的系统，必须明确安全红线。